www.tntextbooks.in

To use a function in the main algorithm, the user need to know only the specification of the function — the function name, the input property, and the input-output relation. The user must ensure that the inputs passed to the function will satisfy the specified property and can assume that the outputs from the function satisfy the input-output relation. Thus, users of the function need only to know what the function does, and not how it is done by the function. The function can be used a a "black box" in solving other problems.

Ultimately, someone implements the function using an algorithm. However, users of the function need not know about the algorithm used to implement the function. It is hidden from the users. There is no need for the users to know how the function is implemented in order to use it.

An algorithm used to implement a function may maintain its own variables. These variables are local to the function in the sense that they are not visible to the user of the function. Consequently, the user has fewer variables to maintain in the main algorithm, reducing the clutter of the main algorithm.

Example 7.7. Consider the problem of testing whether a triangle is right-angled, given its three sides a, b, c, where c is the longest side. The triangle is right-angled, if

$$c^2 = a^2 + b^2$$

We can identify a subproblem of squaring a number. Suppose we have a function square(), specified as

```
square(y)
-- inputs : y
-- outputs : y^2
```

we can use this function three times to test whether a triangle is right-angled. square() is a "black box" — we need not know how the function computes the square. We only need to know its specification.

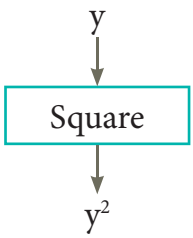

Figure 7.11: square function

```
1  right_angled(a, b, c)
2  -- inputs: c ≥ a, c ≥ b
3  -- outputs: result = true if c² = a² + b²;
4  --              result = false , otherwise
5  if square (c) = square (a) + square (b)
6     result := true
7  else
8     result := false
```